

Nombre:	Documento:
---------	------------

- Durante el examen no se responden preguntas y tiene 1 hora y 45 minutos para resolverlo.
- No puede usar aparatos electrónicos y el examen es individual.
- **Por favor disfrutar el examen y tener confianza en sus conocimientos**
- Tenga en cuenta: “*Qué hubiera sido, si antes ...*” Dario Gómez

Ejercicio 1. (1 punto) Considere la siguiente función:

```
def muy_dificil(S):
    n = len(S)
    A = [0] * n
    for j in range(n):
        total = 0
        for i in range(j + 1):
            total += S[i]
        A[j] = total / (j + 1)
    return A
```

Responder las siguientes preguntas:

- ✓ Hallar el retorno de *muy_dificil*([1,2,3,4])
- ✓ Determine la **complejidad computacional**.

Ejercicio 2. (2 puntos) Responder los siguientes ejercicios:

- ✓ Escriba una función *sumaCola*(*Q*) que reciba una cola de enteros *Q* y retorne la suma de sus elementos. Al finalizar, *Q* debe quedar exactamente igual que al inicio (mismos elementos, mismo orden).

```
def sumaCola(Q):
    ...
```

- ✓ Escriba una función *intercalar*(*Q1*, *Q2*) que reciba dos colas de la **misma longitud** y retorne una **nueva** cola con los elementos intercalados uno a uno (primero el frente de *Q1*, luego el frente de *Q2*, y así sucesivamente). Al finalizar, *Q1* y *Q2* deben quedar **vacías**.

```
def intercalar(Q1, Q2):
    ...
```

Ejercicio 3. (1 punto) Suponga que tiene una pila *S* (*Stack*) con *n* elementos y una cola *Q* (*Queue*) que inicialmente está vacía. Escriba una función *contiene*(*S*, *Q*, *x*) que use *Q* para recorrer *S* y determinar si contiene un elemento *x*, retornando *True* o *False* según corresponda. Al finalizar, *S* debe quedar exactamente en su orden original y *Q* debe quedar vacía. No puede crear listas, ni diccionarios adicionales. Puede crear variables enteras.

```
def contiene(S, Q, x):
    ...
```

Ejercicio 4. (0.75 puntos) Escriba una función **recursiva** `contar_apariciones(lista, x)` que reciba una lista no vacía `lista` y un elemento `x`, y retorne el número de veces que `x` aparece en `lista`. Debe ser recursiva (no puede usar `for/while`, ni el método `count` de las listas).

```
def contar_apariciones(lista, x):
    ...

contar_apariciones([3, 5, 3, 3, 8, 5], 3)
# 3
contar_apariciones([3, 5, 3, 3, 8, 5], 9)
# 0
```

Ejercicio 5. (0.75 puntos) Considere la clase `DynamicArray`:

```
class DynamicArray:
    """A dynamic array class akin to a simplified Python list."""

    def __init__(self):
        """Create an empty array."""
        self._n = 0                # count actual elements
        self._capacity = 1        # default array capacity
        self._A = self._make_array(self._capacity)    # low-level array

    def __len__(self):
        """Return number of elements stored in the array."""
        return self._n

    def __getitem__(self, k):
        """Return element at index k."""
        if not 0 <= k < self._n:
            raise IndexError('invalid index')
        return self._A[k]        # retrieve from array

    def append(self, obj):
        """Add object to end of the array."""
        if self._n == self._capacity:    # not enough room
            self._resize(2 * self._capacity)    # so double capacity
        self._A[self._n] = obj
        self._n += 1

    def _resize(self, c):
        """Resize internal array to capacity c."""
        B = self._make_array(c)        # new (bigger) array
        for k in range(self._n):        # for each existing value
            B[k] = self._A[k]
        self._A = B                    # use the bigger array
        self._capacity = c

    def _make_array(self, c):
        """Return new array with capacity c."""
        return (c * ctypes.py_object)()    # see ctypes documentation
```

- ✓ Agregue a la clase `DynamicArray` un método `reverse(self)` que retorne un **nuevo** `DynamicArray` con los mismos elementos en orden **invertido**, dejando el arreglo original (`self`) **sin modificar**.
- ✓ Se inicia un `DynamicArray` vacío y se realizan agregar cuatro elementos. Hallar el valor de `self._capacity`

```
def reverse(self):
    ...
```